

Week-5 Assignment

Q1: What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Answer:

MapReduce is one of the earliest frameworks used for processing large amounts of data. Although it is reliable, it has several limitations that make Apache Spark a better choice for modern data processing.

The biggest limitation of MapReduce is that it stores intermediate results on the disk after every processing step. Reading and writing data repeatedly to the disk increases execution time, making MapReduce slower, especially for large datasets.

Another limitation is that MapReduce requires writing separate Map and Reduce functions, which makes the programming model more complex. It is also not suitable for iterative tasks like machine learning because every iteration reads data from the disk again.

Apache Spark overcomes these problems by processing data in memory (RAM), which greatly improves speed. Spark also provides easy-to-use APIs such as DataFrames, SQL, and machine learning libraries, making development simpler and more efficient.

Advantages of Spark over MapReduce:

- Faster due to in-memory processing.
- Easy DataFrame and SQL APIs.
- Better support for iterative algorithms.
- Supports real-time streaming and machine learning.
- Less disk I/O, resulting in higher performance.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Answer:

In-memory computing means that Spark stores data in the computer's RAM instead of writing it to the hard disk after every operation.

Machine learning algorithms usually process the same dataset many times before producing the final model. In MapReduce, every iteration reads the data from the disk and writes the results back to the disk, which increases execution time.

Spark keeps the data in memory throughout the iterations. Since RAM is much faster than disk storage, Spark can repeatedly access the data without unnecessary disk operations.

For example, if a machine learning algorithm performs 50 iterations, Spark loads the dataset only once into memory and reuses it for all iterations. This significantly reduces processing time and improves overall performance.

Q3: Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

Answer:

```
from pyspark.sql import SparkSession

# Remove duplicate records based on selected columns
df_clean = df.dropDuplicates(["user_id", "transaction_date"])

# Display the cleaned DataFrame
df_clean.show()
```

Explanation:

The dropDuplicates() function removes duplicate rows based only on the specified columns. If multiple records have the same user_id and transaction_date, Spark keeps only one record and removes the rest. This helps improve data quality before analysis.

Q4: Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Answer:

```
from pyspark.sql.functions import avg, col

result = (
    df_sales
    .filter(col("region") == "West")
```

```
.groupBy("product_category")  
.agg(avg("sale_amount").alias("Average_Sales"))  
)
```

```
result.show()
```

Explanation:

First, the data is filtered to include only records from the **West** region. Then, the data is grouped by product_category, and the average sale amount is calculated for each category.

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

Answer:

Both functions are used to handle missing values, but they work differently.

- **.na.drop()** removes rows that contain null values.
- **.na.fill()** keeps the rows but replaces null values with a specified value.

If the missing data is important, using .na.fill() is usually a better option because it avoids losing records.

Code:

```
df = df.na.fill({"status": "Unknown"})  
df.show()
```

Explanation:

This code replaces every null value in the status column with the text "Unknown".

Q6: Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Answer:

```
from pyspark.sql.functions import count, col
```

```
df.groupBy("city") \
  .agg(count("*").alias("Total_Count")) \
  .filter(col("Total_Count") > 100) \
  .show()
```

Explanation:

The data is grouped by city, the number of records for each city is counted, and only cities with more than 100 records are displayed.

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Answer:

Spark DataFrames are immutable, which means they cannot be modified directly after they are created.

Whenever we perform operations like dropping a column, renaming a column, or filtering data, Spark creates a new DataFrame instead of changing the existing one.

For example:

```
df = df.drop("Age")
```

Here, the original DataFrame is not modified. Instead, Spark creates a new DataFrame without the Age column and stores it in df.

Immutability makes Spark more reliable because the original data remains unchanged and operations become fault-tolerant.

Q8: Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Answer:

```
from pyspark.sql.functions import col
```

```
filtered_df = df.filter(
  (col("age") >= 18) &
```

```
(col("age") <= 30) &  
(col("subscription") == "Premium")  
)
```

```
filtered_df.show()
```

Explanation:

This query selects only those records where the customer's age is between 18 and 30 and the subscription type is "Premium".

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?**Answer:**

Handling null values before performing calculations is important because missing values can produce incorrect or misleading results.

For example, if a column contains many null values, the average may not accurately represent the dataset. Similarly, some calculations may ignore null values completely, while others may produce unexpected results.

By filling missing values or removing incomplete records before aggregation, the calculations become more accurate and reliable.

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.**Answer:**

```
from pyspark.sql.functions import col  
from pyspark.sql.types import TimestampType  
  
df = df.withColumn(  
    "event_time",  
    col("raw_timestamp").cast(TimestampType())
```

```
).drop("raw_timestamp")  
df.show()
```

Explanation:

The raw_timestamp column is converted into the Timestamp data type, and the new column is named event_time. The original column is then removed.

Q11: Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

Answer:

Shuffle is the process of moving data between different partitions in a Spark cluster.

When we perform operations like groupBy() or join(), Spark must collect related records from different partitions. This movement of data across the network is called **Shuffle**.

It is considered a **wide transformation** because data from multiple partitions is exchanged before producing the final result. Shuffle is more expensive than narrow transformations because it involves network communication and disk operations.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Answer:

```
from pyspark.sql.functions import col  
  
clean_df = df.filter(  
    col("email").isNotNull() &  
    (col("username") != "")  
)  
  
clean_df.show()
```

Explanation:

The query keeps only those records where the email column is not null and the username is not empty.

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Answer:

```
from pyspark.sql.functions import min, max, avg
```

```
df.agg(  
    min("price").alias("Minimum_Price"),  
    max("price").alias("Maximum_Price"),  
    avg("price").alias("Average_Price")  
)
```

Explanation:

The .agg() function allows multiple aggregation operations to be performed in a single query, making the code more efficient and easier to read.

Q14: In the context of cleaning a dataset, what is the risk of using inferSchema=True when your source data contains messy or inconsistent date formats?

Answer:

The inferSchema=True option automatically detects the data type of each column.

If a date column contains inconsistent formats, such as:

- 01/05/2024
- 2024-05-01
- May 1, 2024

Spark may incorrectly identify the column as a **String** instead of a **Date**. In some cases, invalid dates may become null values.

This can lead to incorrect filtering, sorting, or date calculations. Therefore, when working with inconsistent data, it is often safer to define the schema manually or clean the date values before loading the dataset.

Q15: Write a final processing pipeline that:

- **Filters out duplicates**
- **Fills null prices with 0**
- **Groups by store_id to calculate total revenue**

Answer:

```
from pyspark.sql.functions import sum
result = (
    df
    .dropDuplicates()
    .fillna({"price": 0})
    .groupBy("store_id")
    .agg(
        sum("price").alias("Total_Revenue")
    )
)
result.show()
```

Explanation:

This pipeline performs three important steps:

1. Removes duplicate records.
2. Replaces missing values in the price column with 0.
3. Groups the data by store_id and calculates the total revenue for each store.

This demonstrates a simple ETL (Extract, Transform, Load) pipeline using Spark DataFrames, which is commonly used in real-world data engineering projects.